

Parameterized Algorithms

Hongseon Im(GIST)



01

Motivation & Background



Bar Fight Prevention problem

Problem Definition:

- You are a security guard. n people plan to come to the bar, and you have a list of pairs who will fight if both are admitted. The management does not want more than k people rejected at the door.
- Question: Can we admit everyone except at most k troublemakers so that no one fights?
- Graph model: guests are vertices, and an edge means "these two will fight."
- Rejected guests form a set that touches every edge, which is exactly a Vertex Cover of size at most k .
- The problem is NP-complete, so no polynomial-time algorithm is expected.



Bar Fight Prevention problem

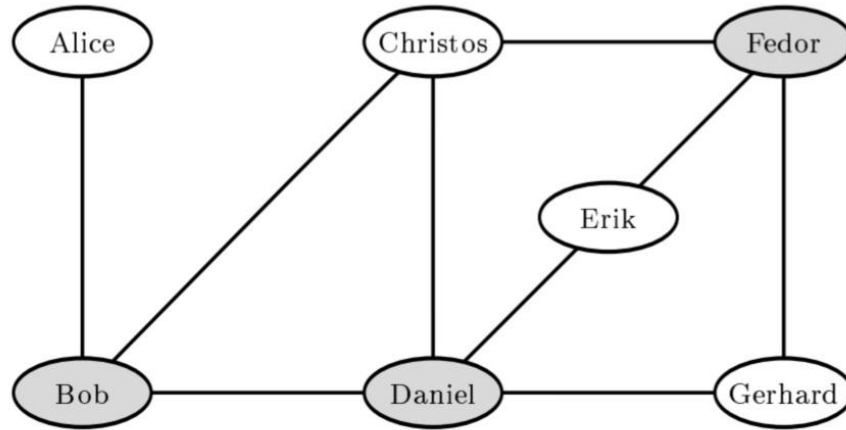


Fig. 1.1: An instance of the BAR FIGHT PREVENTION problem with a solution for $k = 3$. An edge between two guests means that they will fight if both are admitted

Bar Fight Prevention problem

Trivial algorithm:

- Try all subsets of guests to reject. For $n = 1000$ this is $2^{1000} \approx 1.07 \cdot 10^{301}$ possibilities, which is hopeless.
- Use $k \leq 10$: only try subsets of size at most k . This gives $\binom{1000}{10} \approx 2.63 \cdot 10^{23}$ possibilities, better but still infeasible.
- Key question: Can the size of the search space depend on k rather than on n ?



Kernelization

Kernelization: shrinking the input instance

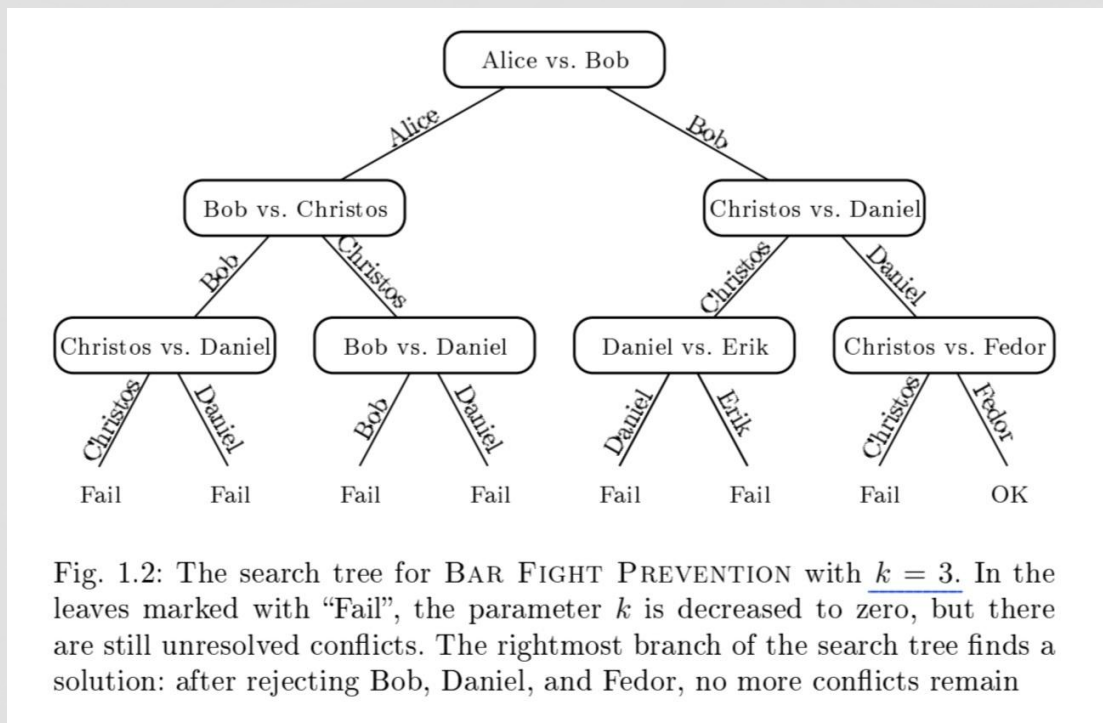
- Rule 1. A guest with no conflicts can be safely admitted.
- Rule 2. A guest with at least $k + 1$ conflicts must be rejected. Otherwise all $k + 1$ opponents would have to be rejected. Reject him and set $k \leftarrow k - 1$.
- Rule 3. A guest with exactly one conflict (Alice vs. Bob) can be admitted, and Bob rejected, with $k \leftarrow k - 1$.
- After exhaustive application, each remaining guest has between 2 and k conflicts.
- If more than k^2 conflicts remain, the answer is no, since each rejection resolves at most k conflicts and we can reject at most k guests. Each undecided guest has at least two conflicts, so there are at most k^2 undecided guests.
- Brute force on the remainder needs at most $\binom{k^2}{k} \leq \binom{100}{10} \approx 1.73 \cdot 10^{13}$ checks, independent of n .

Branching

- Pick any remaining conflict, say Alice vs. Bob. At least one of the two must be rejected.
- Branch 1: reject Alice and recurse with $k - 1$. Branch 2: reject Bob and recurse with $k - 1$.
- Stop when $k = 0$ (succeed if no conflict remains) or when a conflict remains and k has hit 0 (fail).
- The search tree has depth at most k and branching factor 2, hence at most 2^k leaves.
- Running time: $O(2^k \cdot k \cdot n)$.



Branching



FPT vs XP

	FPT	XP
Running time	$f(k) \cdot n^c$	$f(k) \cdot n^{g(k)}$

- FPT: the exponent of n is a constant, and the explosion is confined to $f(k)$.
- XP: $f(k) \cdot n^{g(k)}$. It is polynomial for each fixed k , but the degree grows with k .
- Some problems are not FPT but not NP-hard.
- Tractability depends not only on the problem but also the parameter.



Formal definitions

- Parameterized problem: a language $L \subseteq \Sigma^* \times \mathbb{N}$ over a finite alphabet Σ . For an instance (x, k) , k is the parameter.
- Instance size: $|(x, k)| = |x| + k$.
- FPT: there is an algorithm, a computable function f , and a constant c such that $(x, k) \in L$ is decided in time $f(k) \cdot |(x, k)|^c$.
- XP: the same, but with running time $f(k) \cdot |(x, k)|^{g(k)}$ for computable f, g .
- Solution-size parameters: parameters that are given in the input (ex: k-vertex cover)
- Structural parameters: parameters that measure the instance. (ex: treewidth, degree, genus)



How FPT and mathematics are related

- Mathematics is needed for the proofs of algorithms: König, Hall, Linear Programming
- FPT is usually used for NP-hard problems in combinatorics, graph theory, etc.
- Algorithmic technique proving a theorem: Show that any graph has at most 2^k inclusion-wise minimal vertex covers of size at most k (proved by the search tree, Exercise 3.3).
- branch on an uncovered edge uv (a cover contains u or v), stop at depth k . For a minimal cover C with $|C| \leq k$, follow the branch choosing an endpoint lying in C each time. The leaf is a cover contained in C , hence equal to C by minimality. There are at most 2^k leaves, so at most 2^k such covers.



02

Kernelization



Kernelization

- **Kernelization algorithm (kernel):** an algorithm $\mathcal{A}$ that, given (I, k) , runs in polynomial time and returns an equivalent instance (I', k') , meaning $(I, k) \in Q \Leftrightarrow (I', k') \in Q$.
- Size requirement: $|I'| + k' \leq g(k)$ for some computable function g .
- If g is a polynomial (linear) function, the problem has a **polynomial (linear) kernel**.
- Kernels are built from reduction rules (a rule is safe if it preserves the answer).



Kernelization

- Output size function: $\text{size}_{\mathcal{A}}(k) = \sup\{|I'| + k' : (I', k') = \mathcal{A}(I, k), I \in \Sigma^*\}$.
- A kernelization algorithm is a preprocessing algorithm whose $\text{size}_{\mathcal{A}}(k)$ is finite and bounded by a computable function of k .
- Kernel $\Rightarrow$ FPT: kernelize, then solve the kernel by any algorithm in time $T(g(k))$.
- Lemma 2.2: every FPT problem admits a kernel.
- Proof sketch: run the $f(k) \cdot |I|^c$ algorithm for $|I|^{c+1}$ steps. If it stops, output the answer. Otherwise $f(k) \cdot |I|^c > |I|^{c+1}$, so $|I| < f(k)$, and (I, k) is already a kernel of size at most $f(k) + k$.
- Consequence: a decidable problem has a kernel iff it is FPT. These kernels are usually exponential, which is why problem-specific polynomial kernels matter.



Vertex Cover

- Vertex Cover: given G and k , is there $S \subseteq V(G)$ with $|S| \leq k$ such that every edge has an endpoint in S ?
- **Reduction VC.1.** If G contains an isolated vertex v , delete it. The new instance is $(G - v, k)$.
- Why is this rule safe? An isolated vertex covers no edge, so it is never needed.



Vertex Cover

- **Reduction VC.2.** If some vertex v has degree at least $k + 1$, delete v and decrease k . The new instance is $(G - v, k - 1)$.
- Why safe: if $v \notin S$, then all its $\geq k + 1$ neighbors must be in S , contradicting $|S| \leq k$. So every cover of size at most k contains v .
- After exhaustive use of VC.1 and VC.2, every vertex has degree between 1 and k .



Vertex Cover

- Observation: if the maximum degree is d , then k vertices cover at most kd edges.
- Lemma 2.3. If (G, k) is a yes-instance and neither VC.1 nor VC.2 applies, then $|V(G)| \leq k^2 + k$ and $|E(G)| \leq k^2$.
- Proof: let S be a cover with $|S| \leq k$. No vertex is isolated, so each vertex outside S has a neighbor in S . All degrees are at most k , so $|V(G \setminus S)| \leq k|S|$ and $|V(G)| \leq (k + 1)|S| \leq k^2 + k$. For edges, each vertex of S covers at most k edges, so $|E(G)| \leq k^2$.
- **Reduction VC.3.** If VC.1 and VC.2 no longer apply and G has more than $k^2 + k$ vertices or more than k^2 edges, answer no.
- All rules run in linear time.
- **Theorem 2.4.** Vertex Cover admits a kernel with $O(k^2)$ vertices and $O(k^2)$ edges.



Edge Clique Cover

Problem: given G and k , can the edges of G be covered by at most k cliques?

- Notation: $N(v) = \{u: uv \in E(G)\}$ and $N[v] = N(v) \cup \{v\}$.
- Apply the lowest-numbered rule that changes the instance.
- ECC.1. Remove isolated vertices.
- ECC.2. If uv is an isolated edge (a connected component that is just an edge), delete it and decrease k : the new instance is $(G - \{u, v\}, k - 1)$.
- ECC.3. If $N[u] = N[v]$ for an edge uv , delete v : the new instance is $(G - v, k)$.
- Safeness of ECC.3: a solution for $G - v$ extends to G by adding v to every clique containing u .
- This problem has an exponential-kernel, not a polynomial one.



Edge Clique Cover

Theorem 2.9. Edge Clique Cover admits a kernel with at most 2^k vertices.

- Claim. A reduced yes-instance has at most 2^k vertices.
- Proof: let $C_1, \dots, C_k$ be an edge clique cover. Give each vertex v a vector $b_v \in \{0,1\}^k$ with $b_v[i] = 1$ iff $v \in C_i$. If $|V(G)| > 2^k$, pigeonhole gives $u \neq v$ with $b_u = b_v$.
- If $b_u = b_v = \mathbf{0}$, both are isolated, so ECC.1 applies.
- Otherwise u, v lie in the same cliques, so they are adjacent and $N[u] = N[v]$, so ECC.2 or ECC.3 applies.
- Contradiction with the graph being reduced.
- Algorithm: apply the rules exhaustively. If more than 2^k vertices remain, answer no.



Crown decomposition

Definition 2.10. A crown decomposition of G is a partition $V(G) = C \cup H \cup R$ such that:

- $C \neq \emptyset$.
- C is an independent set.
- There are no edges between C and R (H separates C from R).
- The edges between C and H contain a matching of size $|H|$, i.e. a matching of H into C .
- A matching of U into W saturates U : every vertex of U is an endpoint of a matching edge going to W .

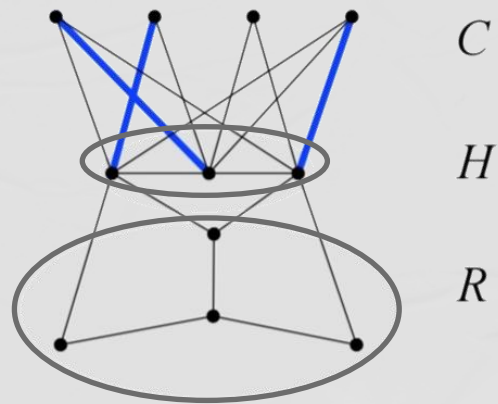


Fig 2.1. Example of a Crown decomposition

Crown decomposition

- Theorem 2.11 (König). In every bipartite graph, the size of a maximum matching equals the size of a minimum vertex cover.
- Theorem 2.12 (Hall). Let G be bipartite with bipartition (V_1, V_2) . G has a matching saturating V_1 iff $|N(X)| \geq |X|$ for every $X \subseteq V_1$.
- Theorem 2.13 (Hopcroft–Karp). For a bipartite graph with n vertices and m edges, a maximum matching and a minimum vertex cover can be found in time $O(m\sqrt{n})$. Also in that time, one finds either a matching saturating V_1 or an inclusion-wise minimal $X \subseteq V_1$ with $|N(X)| < |X|$.



Crown Lemma

Crown Lemma. Let G have no isolated vertices and at least $3k + 1$ vertices. A polynomial-time algorithm either finds a matching of size $k + 1$ or finds a crown decomposition.

- Greedily find a maximal matching M . If $|M| = k + 1$, done. Otherwise $|M| \leq k$, $V_M =$ its endpoints, $|V_M| \leq 2k$, and $I = V(G) \setminus V_M$ is independent (by maximality).
- In the bipartite graph between V_M and I , compute a minimum vertex cover X and maximum matching M' . By König, $|X| = |M'|$; if $|M'| > k$ we again have a matching larger than k . Assume $|X| \leq k$.
- If $X \cap V_M = \emptyset$, then $X \subseteq I$ and $X = I$, so $|V(G)| \leq |I| + |V_M| \leq k + 2k = 3k$, a contradiction.
- Hence $X \cap V_M \neq \emptyset$. Let M^* be the edges of M' with one endpoint in $X \cap V_M$. Set $H = X \cap V_M$, $C = V(M^*) \cap I$, and $R = V(G) \setminus (H \cup C)$. Then C is independent, M^* matches H into C , and since X covers all edges between V_M and I , H separates C from R .



Crown Decomposition $\rightarrow$ Vertex Cover

- Setup. Apply VC.1 to remove isolated vertices. If $|V(G)| > 3k$, apply the Crown Lemma with parameter k .
- Case 1: a matching of size $k + 1$ exists. Answer no, since any vertex cover needs one vertex per matching edge.
- Case 2: a crown decomposition (C, H, R) with matching M of H into C .
- Any cover contains at least $|M| = |H|$ vertices from $H \cup C$.
- The set H alone covers every edge touching $H \cup C$.
- So some minimum cover contains H . Reduce (G, k) to $(G - H, k - |H|)$.
- The vertices of C become isolated and are removed by VC.1. Since $H \neq \emptyset$, the instance strictly shrinks.
- Repeat while $|V(G)| > 3k$.
- **Theorem 2.15.** Vertex Cover admits a kernel with at most $3k$ vertices.



Kernelization via Linear Programming

- Vertex Cover as an integer program
- minimize $\sum_{v \in V(G)} x_v$ subject to $x_u + x_v \geq 1$ for every edge uv , and $x_v \in \{0,1\}$. Relaxing gives $0 \leq x_v \leq 1$.
- There is an optimum solution with all values in $\{0, \frac{1}{2}, 1\}$ (half-integral). Let $V_0, V_{1/2}, V_1$ be the vertices with values $0, \frac{1}{2}, 1$.
- Nemhauser–Trotter (Theorem 2.19). There is a minimum vertex cover S with $V_1 \subseteq S \subseteq V_1 \cup V_{1/2}$.
- **Reduction VC.4.** If $\sum_v x_v > k$, answer no. Otherwise, delete $V_0 \cup V_1$ and decrease k by $|V_1|$.
- What remains has all values equal to $\frac{1}{2}$ with total at most k , so at most $2k$ vertices.
- Theorem 2.21. Vertex Cover admits a kernel with at most $2k$ vertices.
- The LP is solved fast via bipartite matching, so no general-purpose LP solver is needed.



03

Branching



Vertex Cover

- Kernelization plus brute force: the $2k$ -vertex kernel has at most $2^{2k} = 4^k$ subsets to try, giving $O(n\sqrt{m} + 4^k k^{O(1)})$.
- Branching does better. Two observations:
- For any vertex v , a vertex cover contains either v or all of $N(v)$.
- Vertex Cover is trivial (polynomial) when the maximum degree is at most 1.
- Algorithm on (G, k) :
- Pick a vertex v of maximum degree. If $\deg v \leq 1$, every component is an isolated vertex or an edge, and the instance is solved directly.
- Otherwise $|N(v)| \geq 2$. Branch: (a) put v in the cover, delete v , $k \leftarrow k - 1$; (b) put $N(v)$ in the cover, delete $N[v]$, $k \leftarrow k - |N(v)| \leq k - 2$.



Q&A





References

- M. Cygan, F. V. Fomin, Ł. Kowalik, D. Lokshtanov, D. Marx, M. Pilipczuk, M. Pilipczuk, and S. Saurabh, *Parameterized Algorithms*, pp. 1–71. Cham, Switzerland: Springer, 2015.